

app\hw_tmux1204.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      hw_tmux1204.py
4  @brief     Pilote pour multiplexeur TMUX1204 de sélection de résistance de référence.
5  @details   Ce module fournit un pilote pour contrôler le multiplexeur TMUX1204
6              via les signaux de contrôle ADC_A1 et ADC_A0. Permet la sélection
7              de quatre valeurs de résistance de référence (2.2k, 2.7k, 10k, 100k)
8              selon une table de vérité définie.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 from __future__ import annotations
18
19 # Bibliothèque pour la gestion des GPIO
20 from periphery import GPIO
21 # Importation des constantes matérielles
22 from pins_cm5 import GPIO_CHIP_PATH, ADC_A1, ADC_A0
23
24 # -----
25 # Classes principales
26 # -----
27
28 class Tmux1204:
29     """
30     @brief     Pilote pour le multiplexeur TMUX1204.
31     @details   Classe pour gérer la sélection de résistance de référence via
32                 le multiplexeur TMUX1204. Utilise deux signaux de contrôle
33                 (A1, A0) pour sélectionner parmi quatre valeurs de Rref selon
34                 la table de vérité :
35                 - A1=0, A0=0 : 2.2k ohms
36                 - A1=0, A0=1 : 2.7k ohms
37                 - A1=1, A0=0 : 10k ohms
38                 - A1=1, A0=1 : 100k ohms
39     """
40
41     def __init__(self):
42         """
43         @brief     Initialise le pilote TMUX1204.
44         @details   Configure les GPIO pour les signaux de contrôle A1 et A0
45                     en mode sortie. Utilise les constantes matérielles définies
46                     dans le module pins_cm5.
47         """
48         # Initialisation des GPIO pour les sorties A1 et A0
49         # GPIO pour A1
50         self.gpio_a1 = GPIO(GPIO_CHIP_PATH, ADC_A1, "out")
51         # GPIO pour A0
52         self.gpio_a0 = GPIO(GPIO_CHIP_PATH, ADC_A0, "out")
53
54     def close(self):
55         """
56         @brief     Ferme les interfaces GPIO.
57         @details   Ferme proprement les GPIO A1 et A0 pour libérer les
58                     ressources système. Gère les erreurs de fermeture pour
59                     éviter les exceptions lors du nettoyage.
60         """
61         # Fermeture des GPIO
62         try:
63             # Ferme le GPIO A1
64             self.gpio_a1.close()
65         except Exception:
66             # Ignore les erreurs
67             pass
68         try:
69             # Ferme le GPIO A0
70             self.gpio_a0.close()
71         except Exception:
72             # Ignore les erreurs
73             pass
74
75     def set_bits(self, bit_a1, bit_a0):
76         """
77         @brief     Définit les bits de contrôle A1 et A0.
78         @details   Applique les niveaux logiques sur les signaux de contrôle
79                     du multiplexeur. Convertit les valeurs numériques (0/1)
80                     en niveaux logiques GPIO (False/True).
81
82         @param     bit_a1 Valeur du bit A1 (0 ou 1).
83         @param     bit_a0 Valeur du bit A0 (0 ou 1).
84         """

```

```
85         # Définit les bits A1 et A0 (0 ou 1)
86         # Configuration du signal A1
87         if bit_a1 == 0:
88             # Écrit 0 sur A1
89             self.gpio_a1.write(False)
90         else:
91             # Écrit 1 sur A1
92             self.gpio_a1.write(True)
93         # Configuration du signal A0
94         if bit_a0 == 0:
95             # Écrit 0 sur A0
96             self.gpio_a0.write(False)
97         else:
98             # Écrit 1 sur A0
99             self.gpio_a0.write(True)
100
101     def select_rref_ohm(self, rref_ohm):
102         """
103         @brief Sélectionne la résistance de référence Rref.
104         @details Configure les signaux de contrôle selon la table de vérité
105                 pour sélectionner la résistance de référence demandée.
106                 Supporte quatre valeurs fixes : 2200, 2700, 10000, 100000 ohms.
107
108         @param rref_ohm Valeur de résistance de référence en ohms.
109
110         @exception ValueError Si la valeur de Rref n'est pas supportée.
111
112         @note Table de vérité TMUX1204 :
113             - 2200 ohms : A1=0, A0=0
114             - 2700 ohms : A1=0, A0=1
115             - 10000 ohms : A1=1, A0=0
116             - 100000 ohms : A1=1, A0=1
117         """
118         # Sélectionne la résistance de référence Rref
119         # Rref supportées: 2200, 2700, 10000, 100000
120         # Vérification pour 2.2k ohms
121         if rref_ohm == 2200:
122             # A1=0, A0=0 pour 2.2k
123             self.set_bits(0, 0)
124         else:
125             # Vérification pour 2.7k ohms
126             if rref_ohm == 2700:
127                 # A1=0, A0=1 pour 2.7k
128                 self.set_bits(0, 1)
129             else:
130                 # Vérification pour 10k ohms
131                 if rref_ohm == 10000:
132                     # A1=1, A0=0 pour 10k
133                     self.set_bits(1, 0)
134                 else:
135                     # Vérification pour 100k ohms
136                     if rref_ohm == 100000:
137                         # A1=1, A0=1 pour 100k
138                         self.set_bits(1, 1)
139                     else:
140                         # Erreur si Rref non valide
141                         raise ValueError("Rref non supportée: " + str(rref_ohm))
142
```